Arpitha Raichel Jacob
Reg No: 2462607
Class: 5BTELCS

SQL & NoSQL Foundation CIA 1 Practical

SET - 2

Task 1: Hospital Patient Database — Create the Table

Output:

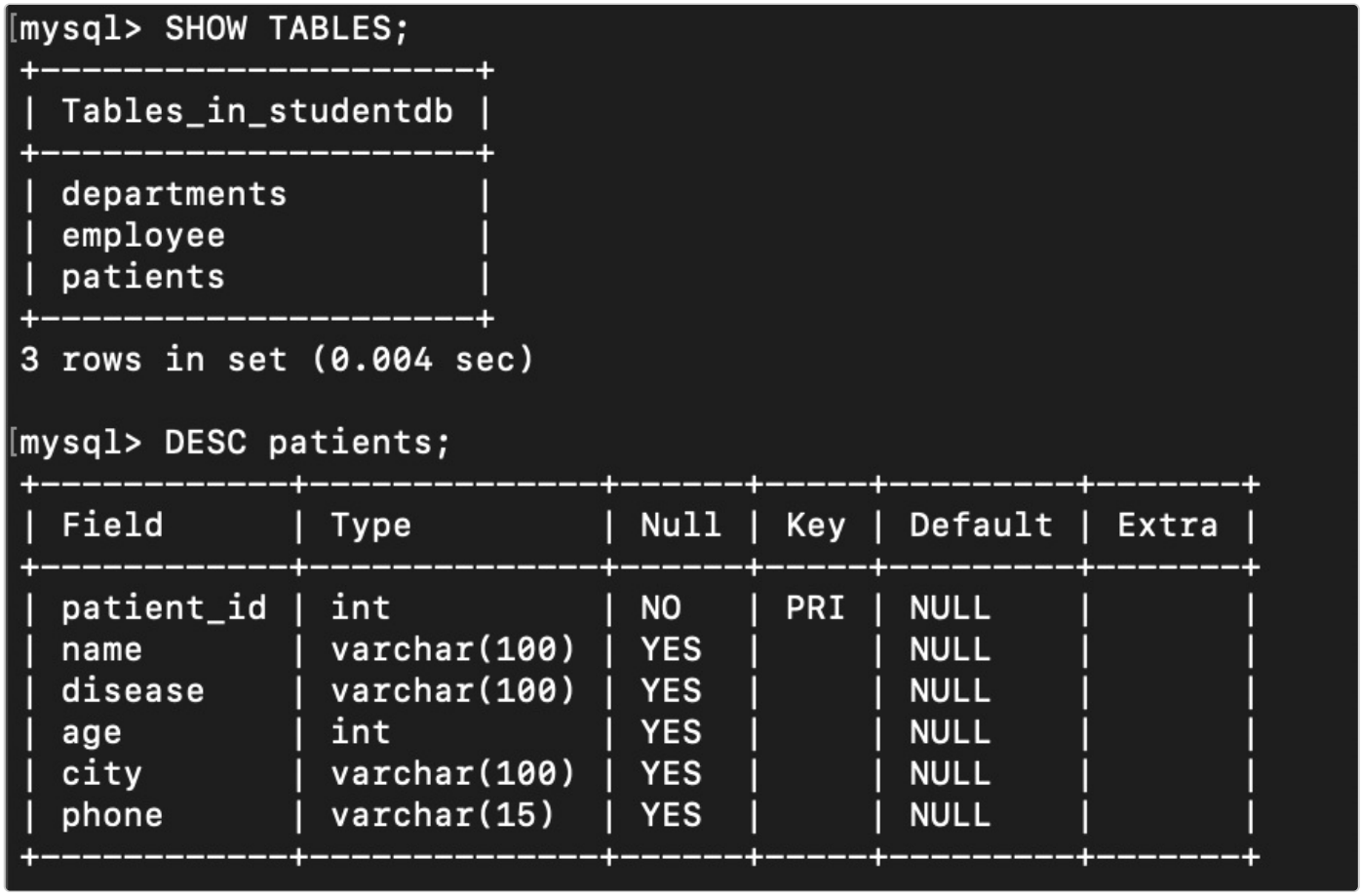
```
mysql> CREATE TABLE patients (
    ->     patient_id INT PRIMARY KEY,
    ->     name VARCHAR(100),
    ->     disease VARCHAR(100),
    ->     age INT,
    ->     city VARCHAR(100),
    ->     phone VARCHAR(15)
    -> );
Query OK, 0 rows affected (0.017 sec)

mysql> SHOW TABLES;
+--------------------+
| Tables_in_studentdb |
+--------------------+
| departments        |
| employee           |
| patients           |
+--------------------+
```

Explanation:

The CREATE TABLE statement successfully builds the patients table with the six required columns, and patient_id is defined as the PRIMARY KEY. The "Query OK, 0 rows affected" message confirms the table was created without errors, and the subsequent SHOW TABLES command lists patients alongside the existing departments and employee tables, proving it now exists in the studentdb database.

Task 2: Hospital Patient Database — Verify the Table Structure

Output:



```
mysql> SHOW TABLES;
+--------------------+
| Tables_in_studentdb |
+--------------------+
| departments        |
| employee           |
| patients           |
+--------------------+
3 rows in set (0.004 sec)

mysql> DESC patients;
+------------+--------------+------+-----+---------+-------+
| Field      | Type         | Null | Key | Default | Extra |
+------------+--------------+------+-----+---------+-------+
| patient_id | int          | NO   | PRI | NULL    |       |
| name       | varchar(100) | YES  |     | NULL    |       |
| disease    | varchar(100) | YES  |     | NULL    |       |
| age        | int          | YES  |     | NULL    |       |
| city       | varchar(100) | YES  |     | NULL    |       |
| phone      | varchar(15)  | YES  |     | NULL    |       |
+------------+--------------+------+-----+---------+-------+
```

Explanation:

SHOW TABLES lists three tables in studentdb, confirming that patients was created successfully. The DESC patients command then displays each column along with its datatype, nullability, and key information, verifying that patient_id is correctly set as the PRIMARY KEY (PRI) while the remaining columns follow the specified structure.

Task 3: Hospital Patient Database — Add a New Column

Output:

```
mysql> ALTER TABLE patients
    -> ADD remarks VARCHAR(255);
Query OK, 0 rows affected (0.019 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql> DESC patients;
+------------+--------------+------+-----+---------+-------+
| Field      | Type         | Null | Key | Default | Extra |
+------------+--------------+------+-----+---------+-------+
| patient_id | int          | NO   | PRI | NULL    |       |
| name       | varchar(100) | YES  |     | NULL    |       |
| disease    | varchar(100) | YES  |     | NULL    |       |
| age        | int          | YES  |     | NULL    |       |
| city       | varchar(100) | YES  |     | NULL    |       |
| phone      | varchar(15)  | YES  |     | NULL    |       |
| remarks    | varchar(255) | YES  |     | NULL    |       |
+------------+--------------+------+-----+---------+-------+
7 rows in set (0.003 sec)
```

Explanation:

The ALTER TABLE ... ADD remarks command adds a new VARCHAR(255) column called remarks to the existing patients table, and the "Query OK" response confirms the change was applied without affecting existing rows. The follow-up DESC patients output now shows seven columns in total, verifying that remarks was appended successfully as a nullable field.

Task 4: Hospital Patient Database — Modify a Column

Output:

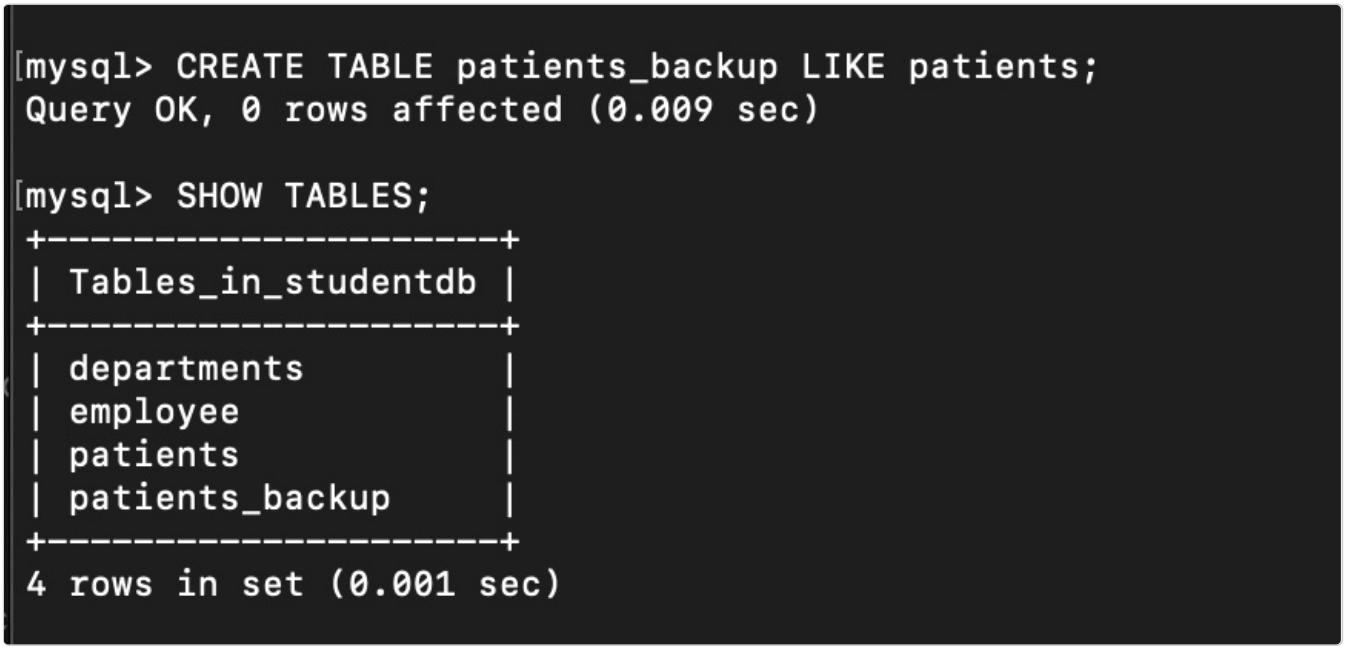
```
mysql> ALTER TABLE patients
    -> MODIFY age TINYINT;
Query OK, 0 rows affected (0.017 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql> DESC patients;
+------------+--------------+------+-----+---------+-------+
| Field      | Type         | Null | Key | Default | Extra |
+------------+--------------+------+-----+---------+-------+
| patient_id | int          | NO   | PRI | NULL    |       |
| name       | varchar(100) | YES  |     | NULL    |       |
| disease    | varchar(100) | YES  |     | NULL    |       |
| age        | tinyint      | YES  |     | NULL    |       |
| city       | varchar(100) | YES  |     | NULL    |       |
| phone      | varchar(15)  | YES  |     | NULL    |       |
| remarks    | varchar(255) | YES  |     | NULL    |       |
+------------+--------------+------+-----+---------+-------+
7 rows in set (0.003 sec)
```

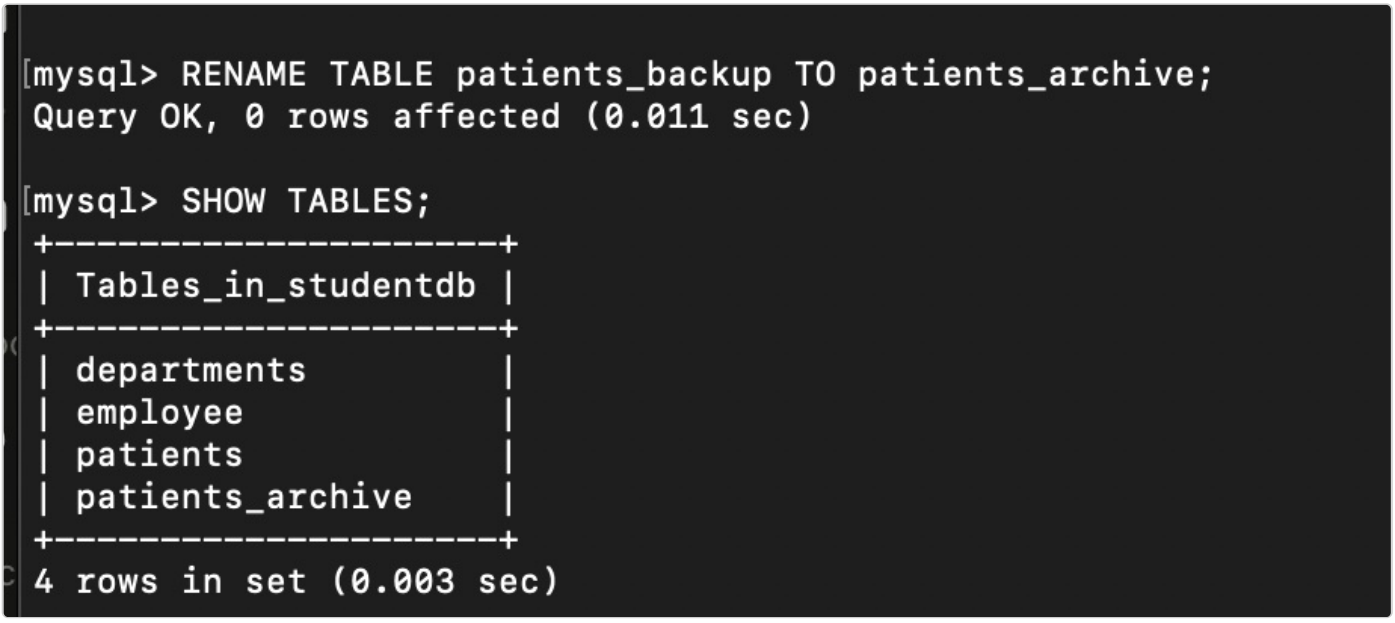
Explanation:

The ALTER TABLE ... MODIFY age TINYINT statement changes the datatype of the age column from INT to TINYINT, which is sufficient since patient ages will never exceed the TINYINT range. The DESC patients output confirms the change, showing age now listed as tinyint while every other column remains unaffected.

Task 5: Hospital Patient Database — Rename the Table

Output:





Explanation:

CREATE TABLE patients_backup LIKE patients duplicates the column structure of patients (without copying data) into a new table, and SHOW TABLES confirms it now appears alongside the original three tables. The RENAME TABLE command then renames patients_backup to patients_archive, and the final SHOW TABLES output verifies that patients_backup no longer exists while patients_archive has taken its place.

Task 6: Hospital Patient Database — TRUNCATE vs DROP

Output:

```
mysql> TRUNCATE TABLE patients_archive;
Query OK, 0 rows affected (0.011 sec)

mysql> SELECT * FROM patients_archive;
Empty set (0.006 sec)

mysql> DROP TABLE patients_archive;
Query OK, 0 rows affected (0.005 sec)

mysql> SHOW TABLES;
+--------------------+
| Tables_in_studentdb |
+--------------------+
| departments        |
| employee           |
| patients           |
+--------------------+
3 rows in set (0.003 sec)
```

Explanation:

TRUNCATE TABLE patients_archive empties all rows while keeping the table structure intact, confirmed by the "Empty set" result of the following SELECT * query. DROP TABLE then permanently removes patients_archive from the database entirely, and the final SHOW TABLES output confirms only three tables (departments, employee, patients) remain, illustrating the difference between clearing data and deleting the whole table.

Task 7: Hospital Patient Database — Insert a Single Row

Output:

```
mysql> INSERT INTO patients
    -> (patient_id,name,disease,age,city,phone,remarks)
    -> VALUES
    -> (101,'Rahul','Fever',25,'Bangalore','9876543210','Stable');
Query OK, 1 row affected (0.003 sec)

mysql> SELECT * FROM patients;
+------------+-------+---------+------+-----------+------------+---------+
| patient_id | name  | disease | age  | city      | phone      | remarks |
+------------+-------+---------+------+-----------+------------+---------+
|        101 | Rahul | Fever   |   25 | Bangalore | 9876543210 | Stable  |
+------------+-------+---------+------+-----------+------------+---------+
1 row in set (0.001 sec)
```

Explanation:

The INSERT INTO patients statement adds one realistic record for patient Rahul, aged 25, from Bangalore with the disease Fever, and the "Query OK, 1 row affected" message confirms the insert succeeded. The subsequent SELECT * FROM patients query displays this single row, verifying that all seven column values were stored correctly.

Task 8: Hospital Patient Database — Insert Multiple Rows

Output:

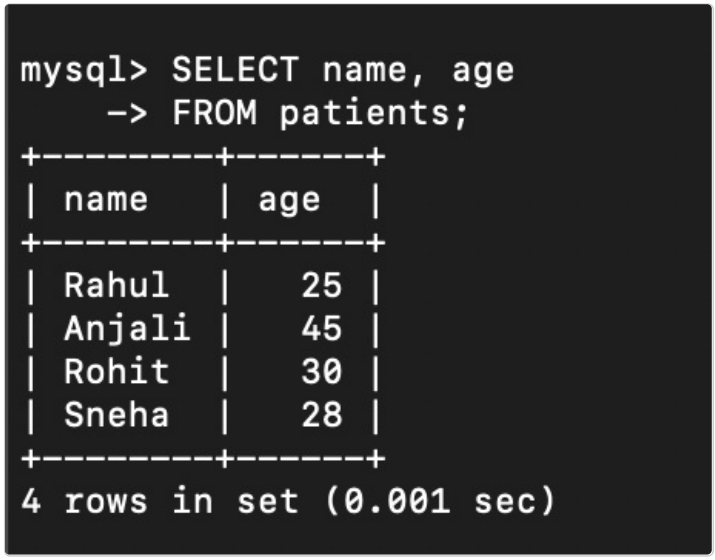
```
mysql> INSERT INTO patients
    -> (patient_id,name,disease,age,city,phone,remarks)
    -> VALUES
    -> (102,'Anjali','Diabetes',45,'Mysore','9876543211','Under Observation'),
    -> (103,'Rohit','Asthma',30,'Chennai','9876543212','Regular Checkup'),
    -> (104,'Sneha','Covid',28,'Hyderabad','9876543213','Recovered');
Query OK, 3 rows affected (0.003 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM patients;
+------------+--------+----------+------+-----------+------------+--------------------+
| patient_id | name   | disease  | age  | city      | phone      | remarks            |
+------------+--------+----------+------+-----------+------------+--------------------+
|        101 | Rahul  | Fever    |   25 | Bangalore | 9876543210 | Stable             |
|        102 | Anjali | Diabetes |   45 | Mysore    | 9876543211 | Under Observation  |
|        103 | Rohit  | Asthma   |   30 | Chennai   | 9876543212 | Regular Checkup    |
|        104 | Sneha  | Covid    |   28 | Hyderabad | 9876543213 | Recovered          |
+------------+--------+----------+------+-----------+------------+--------------------+
4 rows in set (0.001 sec)
```

Explanation:

A single multi-row INSERT statement adds three new patients (Anjali, Rohit, and Sneha) in one query, and "Query OK, 3 rows affected" confirms all three records were inserted together. The SELECT * output now shows all four patients in the table, demonstrating that a multi-row insert is more efficient than running separate INSERT statements for each record.

Task 9: Hospital Patient Database — SELECT All vs Specific Columns

Output:

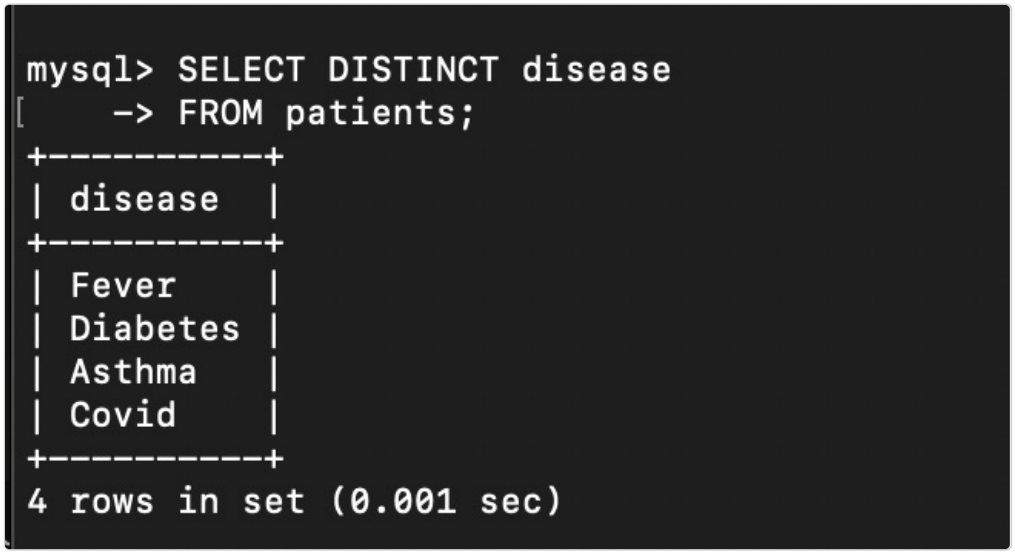


Explanation:

SELECT * FROM patients (shown earlier in Task 8) retrieves every column and row from the table, while SELECT name, age FROM patients shown here retrieves only the two requested columns for all four patients. This comparison highlights how column projection reduces the result set to just the data that's actually needed, rather than returning the entire row.

Task 10: Hospital Patient Database — DISTINCT Values

Output:



Explanation:

SELECT DISTINCT disease FROM patients returns each unique disease value present in the table, namely Fever, Diabetes, Asthma, and Covid. Since none of the four patient records share the same disease, DISTINCT returns all four values, confirming there are currently no duplicate disease entries in the dataset.